

Week 8 Assignment: Diving into Apache Solr

Objective 1 - Conceptual Foundations:

<https://youtu.be/dM-0545k9as>

Solr Architecture:

Solr is an open-source search platform built on Apache Lucene, which handles text indexing and searching. Designed for fast full text search, which traditional databases are not optimized for. Solr runs as an application accessed through REST APIs and a web-based UI, storing data in distributed collections that are split into shards and replicas for scalability and fault tolerance. Using ZooKeeper to manage the cluster, Solr can quickly search across large amounts of data and return results in milliseconds. ZooKeeper allows it to scale to billions of documents while still returning results quickly.

Solr Inverted Index:

An inverted index is the key data structure that makes Solr extremely fast at searching text. Instead of storing documents and then searching through them word by word, Solr flips the process by mapping words to the documents that contain them. When a document is added, Solr breaks the text into words, normalizes them, and stores their locations in the index. This allows Solr to instantly find relevant documents and even support phrase searches, making full-text search fast and efficient even with millions of documents.

The inverted index makes text searches efficient and supports advanced features like phrase searching and relevance ranking, using the same core ideas found in modern cloud search systems like Elasticsearch and OpenSearch.

Example:

A simple way to understand an inverted index is to think about the index at the back of a school textbook. You look up a word, and the index tells you exactly which pages that word appears on. Solr maps words to the documents and locations where they appear, so when you search for a term, Solr can immediately find the matching documents without scanning everything.

Objective 2 - Creating a Solr Collection:

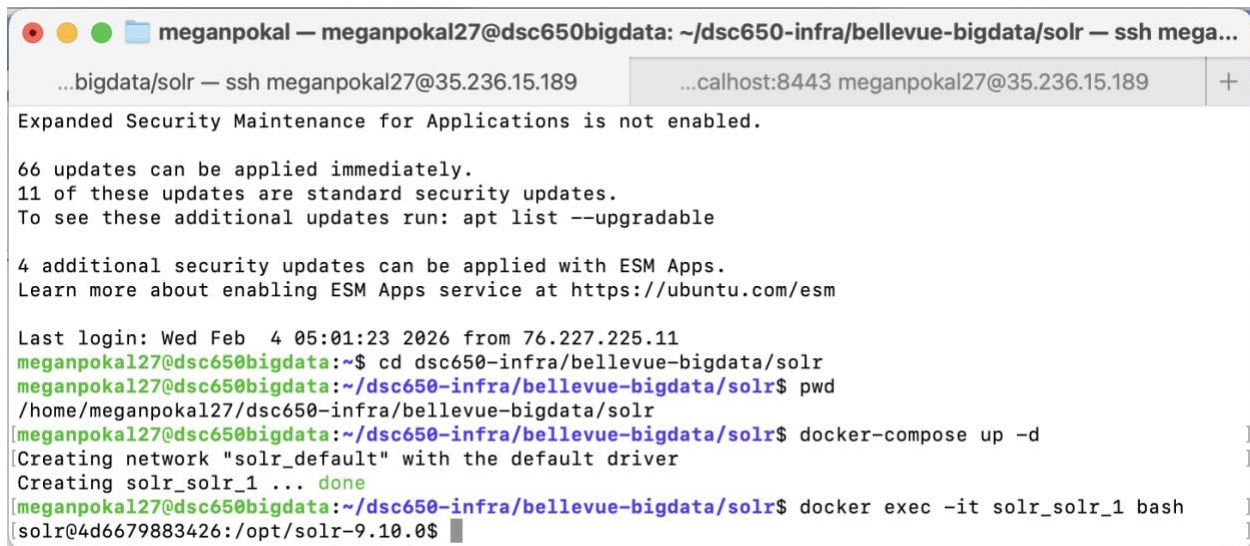
```
ssh -L 9870:localhost:9870 -L 8088:localhost:8088 -L 8080:localhost:8080 -L  
18080:localhost:18080 -L 16010:localhost:16010 -L 8983:localhost:8983 -L 8443:localhost:8443  
meganpokal27@35.236.15.189
```

1. Environment Initialization

- Navigate to the Solr directory:

```
cd dsc650-infra/bellevue-bigdata/solr
```

- Start the Docker containers:
`docker-compose up -d`
- Access the Solr container:
`docker exec -it solr_solr_1 bash`

A terminal window titled 'meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/solr — ssh mega...' shows the following commands and output:

```
...bigdata/solr — ssh meganpokal27@35.236.15.189 ...calhost:8443 meganpokal27@35.236.15.189 +
Expanded Security Maintenance for Applications is not enabled.

66 updates can be applied immediately.
11 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

4 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Wed Feb  4 05:01:23 2026 from 76.227.225.11
meganpokal27@dsc650bigdata:~$ cd dsc650-infra/bellevue-bigdata/solr
meganpokal27@dsc650bigdata:~/dsc650-infra/bellevue-bigdata/solr$ pwd
/home/meganpokal27/dsc650-infra/bellevue-bigdata/solr
[meganpokal27@dsc650bigdata:~/dsc650-infra/bellevue-bigdata/solr$ docker-compose up -d
]
[Creating network "solr_default" with the default driver
]
[Creating solr_solr_1 ... done
]
[meganpokal27@dsc650bigdata:~/dsc650-infra/bellevue-bigdata/solr$ docker exec -it solr_solr_1 bash
]
[solr@4d6679883426:/opt/solr-9.10.0$
```

2. Creating a Solr Collection

In this section, you will create a Solr collection named `mynewcollection`. Solr collections are logical groupings of indexed data, and each collection is capable of being queried independently.

Exercise 1: Create a Solr collection named `'mynewcollection'`.

```
/opt/solr/bin/solr create -c mynewcollection
```

This command creates a new Solr collection named `mynewcollection`. It will be ready to index and store documents.

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/solr — ssh mega...
...bigdata/solr — ssh meganpokal27@35.236.15.189    ...calhost:8443 meganpokal27@35.236.15.189    +
[meganpokal27@dsc650bigdata:~/dsc650-infra/bellevue-bigdata/solr$ docker-compose up -d
Creating network "solr_default" with the default driver
Creating solr_solr_1 ... done
[meganpokal27@dsc650bigdata:~/dsc650-infra/bellevue-bigdata/solr$ docker exec -it solr_solr_1 bash
solr@4d6679883426:/opt/solr-9.10.0$ /opt/solr/bin/solr create -c mynewcollection
Neither --zk-host or --solr-url parameters provided so assuming solr url is http://localhost:8983.
WARNING: Using _default configset. Data driven schema functionality is enabled by default, which is
NOT RECOMMENDED for production use.

To turn it off:
  curl http://localhost:8983/solr/mynewcollection/config -d '{"set-user-property": {"update.
autoCreateFields":"false"}}'
Or:
  bin/solr config -c mynewcollection -s http://localhost:8983 --action set-user-property --p
roperty update.autoCreateFields --value false

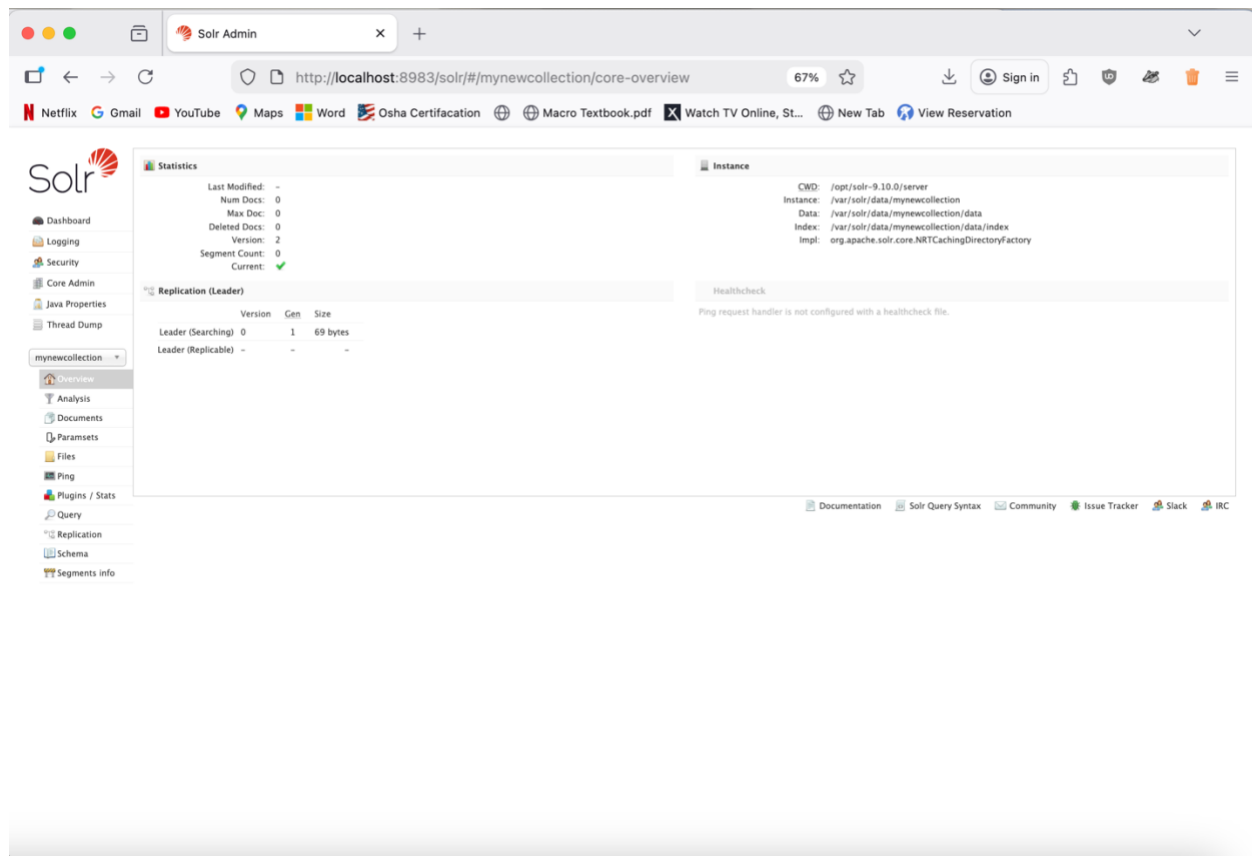
Created new core 'mynewcollection'
solr@4d6679883426:/opt/solr-9.10.0$
```

Exercise 2: Verify that ‘mynewcollection’ has been successfully created.

- Access Solr’s Web Interface at <http://localhost:8983/solr/> and check for ‘mynewcollection’ under the “Core Selector” dropdown.

The “mynewcollection’ on Solr local host has been successfully created.

<http://localhost:8983/solr/#/mynewcollection/core-overview>, and appears in the Solr Web interface dropdown. Being listed indicates that Solr has initialized the collection and is it ready to accept data.



Objective 3 - Adding Data to the Collection:

1. Generating and Adding Data to the Collection

You will now generate sample data in JSON format and add it to the mynewcollection for indexing. This step allows you to see how Solr ingests structured data into its indexing engine.

For this exercise, create a json file called products.json using this commands:

```
echo '[
  {"id":"1", "name":"Product A", "category":"Electronics",
"price":"100"},
  {"id":"2", "name":"Product B", "category":"Books", "price":"20"}
]' > /tmp/products.json
```

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/solr — ssh mega...
...bigdata/solr — ssh meganpokal27@35.236.15.189 ...calhost:8443 meganpokal27@35.236.15.189 +

Created new core 'mynewcollection'
solr@d6679883426:/opt/solr-9.10.0$ echo '[
  {"id": "1", "name": "Product A", "category": "Electronics", "price": "100"},
  {"id": "2", "name": "Product B", "category": "Books", "price": "20"}
]' > /tmp/products.json
solr@d6679883426:/opt/solr-9.10.0$
```

Exercise 3: Let's add the generated data to our collection.

```
/opt/solr/bin/solr post -c mynewcollection /tmp/products.json
```

Above you added information and added it to a file called products.json. This file has been successfully added to into mynewcollection. This commend ingests the JSON data into mynewcollection.

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/solr — ssh mega...
...bigdata/solr — ssh meganpokal27@35.236.15.189 ...calhost:8443 meganpokal27@35.236.15.189 +

Created new core 'mynewcollection'
solr@d6679883426:/opt/solr-9.10.0$ echo '[
  {"id": "1", "name": "Product A", "category": "Electronics", "price": "100"},
  {"id": "2", "name": "Product B", "category": "Books", "price": "20"}
]' > /tmp/products.json
solr@d6679883426:/opt/solr-9.10.0$ /opt/solr/bin/solr post -c mynewcollection /tmp/products.json
Posting files to [base] url http://localhost:8983/solr/mynewcollection/update...
Entering auto mode. File endings considered are xml,json,jsonl,csv,pdf,doc,docx,ppt,pptx,xls,xlsx,odt,
odp,ods,ott,otp,ots,rtf,htm,html,txt,log
POSTing file products.json (application/json) to [base]/json/docs
1 files indexed.
COMMITting Solr index changes to http://localhost:8983/solr/mynewcollection/update...
WARNING: URLs provided to this tool needn't include Solr's context-root (e.g. "/solr"). Such URLs are
deprecated and support for them will be removed in a future release. Correcting from [http://localhost
:8983/solr] to [http://localhost:8983/].
Time spent: 0:00:00.812
solr@d6679883426:/opt/solr-9.10.0$
```

The screenshot shows the Solr Admin web interface in a browser. The address bar displays `http://localhost:8983/solr/#/mynewcollection/core-overview`. The interface is divided into several sections:

- Statistics:** Shows collection metrics: Last Modified (2 minutes ago), Num Docs (2), Max Doc (2), Deleted Docs (0), Version (6), Segment Count (1), and Current (green checkmark).
- Replication (Leader):** A table showing replication status for the leader node.
- Instance:** Displays system paths: CWD, Instance, Data, Index, and Impl.
- Healthcheck:** A message stating "Ping request handler is not configured with a healthcheck file."

	Version	Gen	Size
Leader (Searching)	1770766508888	2	4.47 KB
Leader (Replicable)	1770766508888	2	-

Objective 4 - Querying the Collection:

1. Querying the Collection

Now that the data has been ingested, you will perform queries on the mynewcollection using Solr's command line query interface. This demonstrates how Solr processes and retrieves indexed data.

Exercise 4: Use Solr's query interface to retrieve all documents from 'mynewcollection'.

```
curl http://localhost:8983/solr/mynewcollection/select?q=*:*
```

This query shows all documents from mynewcollection. This confirms that the data was successfully put into Solr and is searchable. `Q=*:*` is a match all query.

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/solr — ssh mega...
...bigdata/solr — ssh meganpokal27@35.236.15.189    ...calhost:8443 meganpokal27@35.236.15.189    +
Time spent: 0:00:00.812
solr@4d6679883426:/opt/solr-9.10.0$ curl "http://localhost:8983/solr/mynewcollection/select?q=:*:*"
{
  "responseHeader":{
    "status":0,
    "QTime":52,
    "params":{
      "q":":*:*"
    }
  },
  "response":{
    "numFound":2,
    "start":0,
    "numFoundExact":true,
    "docs":[{
      "id":"1",
      "name":["Product A"],
      "category":["Electronics"],
      "price":[100],
      "_version_":1856783262366564352,
      "_root_":"1"
    },{
      "id":"2",
      "name":["Product B"],
      "category":["Books"],
      "price":[20],
      "_version_":1856783262445207552,
      "_root_":"2"
    }]
  }
}
solr@4d6679883426:/opt/solr-9.10.0$
```

Exercise 5: Query for products in the 'Electronics' category.

```
curl http://localhost:8983/solr/mynewcollection/select?q=category:Electronics
```

This query shows documents from mynewcollection where the category field is Electronics. Showing only the products classified under Electronics. Solr can filter by specific field values.

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/solr — ssh mega...
...bigdata/solr — ssh meganpokal27@35.236.15.189 ...calhost:8443 meganpokal27@35.236.15.189 +
}
[solr@4d6679883426:/opt/solr-9.10.0$ curl "http://localhost:8983/solr/mynewcollection/select?q=category:Electronics"
{
  "responseHeader":{
    "status":0,
    "QTime":19,
    "params":{
      "q":"category:Electronics"
    }
  },
  "response":{
    "numFound":1,
    "start":0,
    "numFoundExact":true,
    "docs":[{
      "id":"1",
      "name":["Product A"],
      "category":["Electronics"],
      "price":[100],
      "_version_":1856783262366564352,
      "_root_":"1"
    }]
  }
}
solr@4d6679883426:/opt/solr-9.10.0$ █
```

Objective 5 - Advanced Querying in the Solr Web Interface:

1. Advanced Querying in Solr Web Interface

Access the Solr Web Interface for querying:

Query 'mynewcollection' in Solr Web Interface:

Exercise 6: Run a faceted search on the id field to count how many documents are in your collection.

All the documents in mynewcollection. A facet_counts section shows each unique id value and how many documents are associated with it. This faceted search groups documents by id field and displays the count for each unique id. Showing how many documents exist in the collection and confirms that each document is indexed correctly.

The screenshot shows the Solr Admin web interface in a browser. The address bar displays `http://localhost:8983/solr/`. The left sidebar contains navigation links: Dashboard, Logging, Security, Core Admin, Java Properties, Thread Dump, mynewcollection (selected), Overview, Analysis, Documents, Paramsets, Files, Ping, Plugins / Stats, Query (selected), Replication, Schema, and Segments info.

The main panel is titled "Request-Handler (qt)" and shows the configuration for the `/select` handler. The configuration includes:

- `q`: `*:*`
- `q.op`: `OR`
- `fq`: (empty)
- `sort`: (empty)
- `start, rows`: `0` to `10`
- `fl`: (empty)
- `df`: (empty)
- `paramset(s)`: `Select paramset(s)...`
- `wt`: `json`
- `indent on`: ☒
- `debugQuery`: ☐
- `defType`: `lucene`
- `hl`: ☐
- `facet`: ☒
- `facet.query`: (empty)
- `facet.field`: `id`
- `facet.prefix`: (empty)
- `facet.contains`: (empty)
- `facet.contains.ignoreCase`: ☐
- `facet.limit`: (empty)

The right panel displays the JSON response for the query `http://localhost:8983/solr/mynewcollection/select?facet.field=id&facet=true&indent=true&q.op=OR&q=%3A*&use`:

```
{
  "responseHeader": {
    "status": 0,
    "QTime": 14,
    "params": {
      "q": "*:*",
      "facet.field": "id",
      "indent": "true",
      "q.op": "OR",
      "facet": "true",
      "useParams": "",
      "_": "1770767065719"
    }
  },
  "response": {
    "numFound": 2,
    "start": 0,
    "numFoundExact": true,
    "docs": [
      {
        "id": "1",
        "name": ["Product A"],
        "category": ["Electronics"],
        "price": [100],
        "_version_": 1856783262366564352,
        "_root_": "1"
      },
      {
        "id": "2",
        "name": ["Product B"],
        "category": ["Books"],
        "price": [20],
        "_version_": 1856783262445207552,
        "_root_": "2"
      }
    ]
  },
  "facet_counts": {
    "facet_queries": { },
    "facet_fields": {
      "id": ["1", 1, "2", 1]
    },
    "facet_ranges": { },
    "facet_intervals": { },
    "facet_heatmaps": { }
  }
}
```

Exercise 7: Use the Web Interface to experiment further with various query parameters and filters.

Query 1: Sorting results by price

Sort: price asc

This query sorts all the documents by the price field in ascending order. The results show that Product B, price 20, appears before Product A, price 100, confirming that Solr successfully put them from lowest to highest price.

The screenshot shows the Solr Admin web interface in a browser. The address bar displays `http://localhost:8983/solr/`. The left sidebar contains navigation links: Dashboard, Logging, Security, Core Admin, Java Properties, Thread Dump, mynewcollection (selected), Overview, Analysis, Documents, Paramsets, Files, Ping, Plugins / Stats, Query (selected), Replication, Schema, and Segments info.

The main content area is titled "Request-Handler (qt)" and shows the configuration for the `/select` handler. The configuration includes:

- q: `*:*`
- q.op: `OR`
- fq: (empty)
- sort: `price asc`
- start, rows: `0` and `10`
- fl: (empty)
- df: (empty)
- paramset(s): `Select paramset(s)...`
- wt: `json`
- ☒ indent on
- ☐ debugQuery
- defType: `lucene`
- ☐ hl
- ☐ facet
- ☐ spatial
- ☐ spellcheck
- Raw Query Parameters: (empty)
- JSON Query: (empty)

The "Execute Query" button is at the bottom of the configuration panel.

The right panel displays the JSON response for the query `http://localhost:8983/solr/mynewcollection/select?indent=true&q.op=OR&q=%3A*&sort=price%20asc&useParams=`:

```
{
  "responseHeader": {
    "status": 0,
    "QTime": 15,
    "params": {
      "q": "*:*",
      "indent": "true",
      "q.op": "OR",
      "sort": "price asc",
      "useParams": "",
      "_: "1770767065719"
    }
  },
  "response": {
    "numFound": 2,
    "start": 0,
    "numFoundExact": true,
    "docs": [
      {
        "id": "2",
        "name": ["Product B"],
        "category": ["Books"],
        "price": [20],
        "_version_": 1856783262445207552,
        "_root_": "2"
      },
      {
        "id": "1",
        "name": ["Product A"],
        "category": ["Electronics"],
        "price": [100],
        "_version_": 1856783262366564352,
        "_root_": "1"
      }
    ]
  }
}
```

At the bottom of the interface, there are links for Documentation, Solr Query Syntax, Community, Issue Tracker, Slack, and IRC.

Query 2:

q: `*:*`

Start, Rows: 0, 5

This query limits the results to five documents at a time. Even though this collection only has two documents, it shows how Solr supports paging through large datasets by your desired results. This demonstrates pagination by using the rows 5 and start 0.

The screenshot displays the Solr Admin web interface in a browser. The address bar shows the URL `http://localhost:8983/solr/`. The left sidebar contains navigation links: Dashboard, Logging, Security, Core Admin, Java Properties, Thread Dump, mynewcollection (selected), Overview, Analysis, Documents, Paramsets, Files, Ping, Plugins / Stats, Query (selected), Replication, Schema, and Segments info.

The main content area is divided into two panels. The left panel, titled "Request-Handler (qt)", shows the configuration for the `/select` handler. It includes fields for `q` (set to `*:*`), `q.op` (set to `OR`), `fq`, `sort`, `start, rows` (set to `0, 5`), `fl`, `df`, `paramset(s)`, `wt` (set to `json`), and `indent on` (checked). There are also checkboxes for `debugQuery`, `hl`, `facet`, `spatial`, and `spellcheck`. The right panel displays the JSON response for the query `http://localhost:8983/solr/mynewcollection/select?indent=true&q.op=OR&q=%3A*&rows=5&start=0&useParams=`. The response is a JSON object with a `responseHeader` and a `response` array containing two document objects.

```
{
  "responseHeader": {
    "status": 0,
    "QTime": 1,
    "params": {
      "q": "*:*",
      "indent": "true",
      "start": "0",
      "q.op": "OR",
      "rows": "5",
      "useParams": "",
      "_": "1770767065719"
    }
  },
  "response": {
    "numFound": 2,
    "start": 0,
    "numFoundExact": true,
    "docs": [
      {
        "id": "1",
        "name": ["Product A"],
        "category": ["Electronics"],
        "price": [100],
        "_version_": 1856783262366564352,
        "_root_": "1"
      },
      {
        "id": "2",
        "name": ["Product B"],
        "category": ["Books"],
        "price": [20],
        "_version_": 1856783262445207552,
        "_root_": "2"
      }
    ]
  }
}
```

At the bottom of the interface, there are links to Documentation, Solr Query Syntax, Community, Issue Tracker, Slack, and IRC.

Objective 6 - Working with Custom JSON Data:

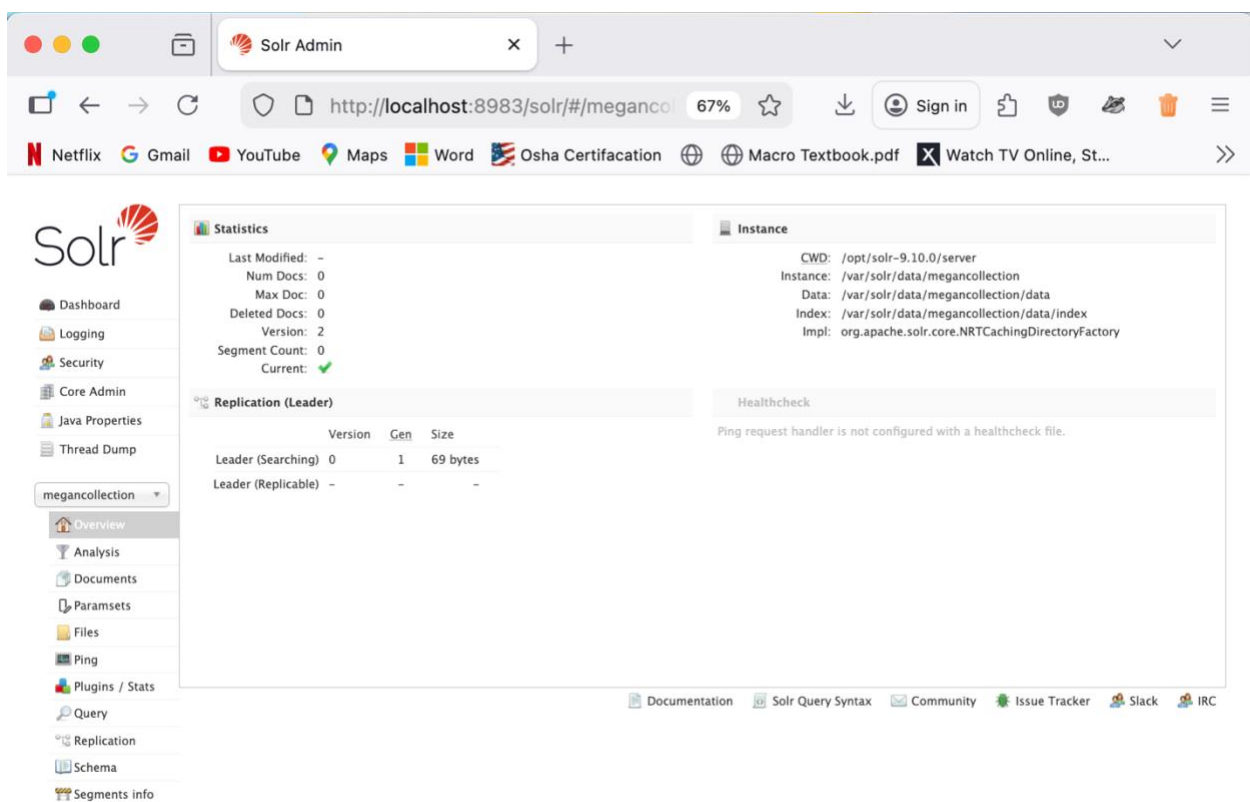
1. Create a new collection

```
bin/solr create -c megancollection
```

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/solr — ssh mega...
...bigdata/solr — ssh meganpokal27@35.236.15.189 ...calhost:8443 meganpokal27@35.236.15.189 +
solr@4d6679883426:/opt/solr-9.10.0$ bin/solr create -c megancollection
Neither --zk-host or --solr-url parameters provided so assuming solr url is http://localhost:8983.
WARNING: Using _default configset. Data driven schema functionality is enabled by default, which is
NOT RECOMMENDED for production use.

To turn it off:
    curl http://localhost:8983/solr/megancollection/config -d '{"set-user-property": {"update.
autoCreateFields":"false"}}'
Or:
    bin/solr config -c megancollection -s http://localhost:8983 --action set-user-property --p
roperty update.autoCreateFields --value false

Created new core 'megancollection'
solr@4d6679883426:/opt/solr-9.10.0$
```



2. Use JSON data from the web
Creating a json file.
Website: <https://developers.squarespace.com/view-json-data>

```
echo '{
{
  "id": "1",
  "collection_id": "5048fde7c4aa917cbd4d8e13",
```

```
"websiteId": "50295e80e4b096e761d7e4d3",
"enabled": true,
"starred": false,
"type": 1,
"ordering": 2,
"title": "Blog",
"navigationTitle": "Blog",
"urlId": "blog",
"itemCount": 2,
"updatedAt": 1454432700858,
"pageSize": 20,
"folder": false,
"dropdown": false,
"tags": ["tag1", "pizza", "coffee", "snacks", "tag2"],
"categories": ["category1", "category2"],
"homepage": false,
"typeName": "blog",
"synchronizing": false,
"fullUrl": "/blog/",
"passwordProtected": false,
"typeLabel": "blog"
},
{
  "id": "2",
  "collection_id": "5048fde7c4aa917cbd4d8e13",
  "websiteId": "50295e80e4b096e761d7e4d3",
  "enabled": true,
  "starred": true,
  "type": 1,
  "ordering": 3,
  "title": "Food Blog",
  "navigationTitle": "Food Blog",
  "urlId": "food-blog",
  "itemCount": 5,
  "updatedAt": 1454432709999,
  "pageSize": 20,
  "folder": false,
  "dropdown": false,
  "tags": ["pizza", "coffee"],
  "categories": ["category2"],
  "homepage": false,
  "typeName": "blog",
  "synchronizing": false,
  "fullUrl": "/food-blog/",
```

```

    "passwordProtected": false,
    "typeLabel": "blog"
  }
]' > /tmp/products.json

```

After finding this sample JSON dataset from the web, we have to create a json file called products.json using the command above. This will allow the information to be in the file products.json, in megancollection.

The screenshot shows a terminal window with the following content:

```

meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/solr — ssh meg...
...igdata/solr — ssh meganpokal27@35.236.15.189    ...calhost:8443 meganpokal27@35.236.15.189    +
solr@4d6679883426:/opt/solr-9.10.0$ echo '[
{
  "id": "1",
  "collection_id": "5048fde7c4aa917cbd4d8e13",
  "websiteId": "50295e80e4b096e761d7e4d3",
  "enabled": true,
  "starred": false,
  "type": 1,
  "ordering": 2,
  "title": "Blog",
  "navigationTitle": "Blog",
  "urlId": "blog",
  "itemCount": 2,
  "updatedOn": 1454432700858,
  "pageSize": 20,
  "folder": false,
  "dropdown": false,
  "tags": ["tag1", "pizza", "coffee", "snacks", "tag2"],
  "categories": ["category1", "category2"],
  "homepage": false,
  "typeName": "blog",
  "synchronizing": false,
  "fullUrl": "/blog/",
  "passwordProtected": false,
  "typeLabel": "blog"
},
{
  "id": "2",
  "collection_id": "5048fde7c4aa917cbd4d8e13",
  "websiteId": "50295e80e4b096e761d7e4d3",
  "passwordProtected": false,
  "typeLabel": "blog"
}]' > /tmp/products.jsonfalse,,g",
solr@4d6679883426:/opt/solr-9.10.0$ bin/solr post -c megancollection /tmp/products.json

```

```
bin/solr post -c megancollection /tmp/products.json
```

Adding the generated data to the new collection.

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/solr — ssh meg...
...igdata/solr — ssh meganpokal27@35.236.15.189 ...calhost:8443 meganpokal27@35.236.15.189 +
[solr@4d6679883426:/opt/solr-9.10.0$ bin/solr post -c megancollection /tmp/products.json
Posting files to [base] url http://localhost:8983/solr/megancollection/update...
Entering auto mode. File endings considered are xml,json,jsonl,csv,pdf,doc,docx,ppt,pptx,xls,xlsx,odt
,odp,ods,ott,otp,ots,rtf,htm,html,txt,log
POSTing file products.json (application/json) to [base]/json/docs
1 files indexed.
COMMITting Solr index changes to http://localhost:8983/solr/megancollection/update...
WARNING: URLs provided to this tool needn't include Solr's context-root (e.g. "/solr"). Such URLs are
deprecated and support for them will be removed in a future release. Correcting from [http://localho
st:8983/solr] to [http://localhost:8983/].
Time spent: 0:00:00.700
solr@4d6679883426:/opt/solr-9.10.0$
```

Query 1: to retrieve all documents from megancollection

```
curl http://localhost:8983/solr/megancollection/select?q=*:*
```

This query shows all documents from the megancollection using the match-all query. The results confirm that the JSON dataset was successfully indexed into Solr and the documents are searchable.


```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/solr — ssh meg...
...igdata/solr — ssh meganpokal27@35.236.15.189 ...calhost:8443 meganpokal27@35.236.15.189 +
[solr@4d6679883426:/opt/solr-9.10.0$ curl "http://localhost:8983/solr/megancollection/select?q=:*" ]
{
  "responseHeader":{
    "status":0,
    "QTime":0,
    "params":{
      "q":":*"
    }
  },
  "response":{
    "numFound":2,
    "start":0,
    "numFoundExact":true,
    "docs":[{
      "id":"1",
      "collection_id":["5048fde7c4aa917cbd4d8e13"],
      "websiteId":["50295e80e4b096e761d7e4d3"],
      "enabled":[true],
      "starred":[false],
      "type":[1],
      "ordering":[2],
      "title":["Blog"],
      "navigationTitle":["Blog"],
      "urlId":["blog"],
      "itemCount":[2],
      "updatedOn":[1454432700858],
      "pageSize":[20],
      "folder":[false],
      "dropdown":[false],
      "tags":["tag1", "pizza", "coffee", "snacks", "tag2"],
      "categories":["category1", "category2"],
      "homepage":[false],
      "typeName":["blog"],
      "synchronizing":[false],
      "fullUrl":["/blog/"],
      "passwordProtected":[false],
      "typeLabel":["blog"],
      "_version_":1856787631098036224,
      "_root_":"1"
    },{
      "id":"2",
      "collection_id":["5048fde7c4aa917cbd4d8e13"],
      "websiteId":["50295e80e4b096e761d7e4d3"],
      "enabled":[true],
      "starred":[true],
      "type":[1],
      "ordering":[3],
      "title":["Food Blog"],
      "navigationTitle":["Food Blog"],
      "urlId":["food-blog"],
      "itemCount":[5],
      "updatedOn":[1454432709999],
      "pageSize":[20],
      "folder":[false],
```

Query 2: Search by Tag Pizza

curl <http://localhost:8983/solr/megancollection/select?q=tags:pizza>

This query searches for documents containing the tag pizza. Solr successfully indexed the fields and can return documents based on specific tag values.


```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/solr — ssh meganpokal27...
...vue-bigdata/solr — ssh meganpokal27@35.236.15.189
...43:localhost:8443 meganpokal27@35.236.15.189 +
}
solr@4d6679883426:/opt/solr-9.10.0$ curl "http://localhost:8983/solr/megancollection/select?q=tags:pizza"
{
  "responseHeader":{
    "status":0,
    "QTime":2,
    "params":{
      "q":"tags:pizza"
    }
  },
  "response":{
    "numFound":2,
    "start":0,
    "numFoundExact":true,
    "docs":[{
      "id":"2",
      "collection_id":["5048fde7c4aa917cbd4d8e13"],
      "websiteId":["50295e80e4b096e761d7e4d3"],
      "enabled":[true],
      "starred":[true],
      "type":1,
      "ordering":3,
      "title":["Food Blog"],
      "navigationTitle":["Food Blog"],
      "urlId":["food-blog"],
      "itemCount":5,
      "updatedOn":1454432709999,
      "pageSize":20,
      "folder":false,
      "dropdown":false,
      "tags":["pizza","coffee"],
      "categories":["category2"],
      "homepage":false,
      "typeName":["blog"],
      "synchronizing":false,
      "fullUrl":["/food-blog/"],
      "passwordProtected":false,
      "typeLabel":["blog"],
      "_version_":1856787631105376256,
      "_root_":"2"
    },{
      "id":"1",
      "collection_id":["5048fde7c4aa917cbd4d8e13"],
      "websiteId":["50295e80e4b096e761d7e4d3"],
      "enabled":[true],
      "starred":false,
      "type":1,
      "ordering":2,
      "title":["Blog"],
      "navigationTitle":["Blog"],
      "urlId":["blog"],
      "itemCount":2,
      "updatedOn":1454432700858,
      "pageSize":20,
    }
  ]
}
```

Query 3: Filter

curl <http://localhost:8983/solr/megancollection/select?q=starred:true>

This query shows only documents where the starred field is true. It also shows Solr ability to filter documents using Boolean field values.

```
meganpokal — meganpokal27@dsc650bigdata: ~/dsc650-infra/bellevue-bigdata/solr — ssh meganpokal27...
...vue-bigdata/solr — ssh meganpokal27@35.236.15.189    ...43:localhost:8443 meganpokal27@35.236.15.189    +
}
solr@4d6679883426:/opt/solr-9.10.0$ curl http://localhost:8983/solr/megancollection/select?q=starred:true
{
  "responseHeader":{
    "status":0,
    "QTime":1,
    "params":{
      "q":"starred:true"
    }
  },
  "response":{
    "numFound":1,
    "start":0,
    "numFoundExact":true,
    "docs":[{
      "id":"2",
      "collection_id":["5048fde7c4aa917cbd4d8e13"],
      "websiteId":["50295e80e4b096e761d7e4d3"],
      "enabled":[true],
      "starred":[true],
      "type":[1],
      "ordering":[3],
      "title":["Food Blog"],
      "navigationTitle":["Food Blog"],
      "urlId":["food-blog"],
      "itemCount":[5],
      "updatedOn":["1454432709999"],
      "pageSize":[20],
      "folder":[false],
      "dropdown":[false],
      "tags":["pizza","coffee"],
      "categories":["category2"],
      "homepage":[false],
      "typeName":["blog"],
      "synchronizing":[false],
      "fullUrl":["/food-blog/"],
      "passwordProtected":[false],
      "typeLabel":["blog"],
      "_version_":1856787631105376256,
      "_root_":"2"
    }]
  }
}
solr@4d6679883426:/opt/solr-9.10.0$
```

Query 4:

Q: tags: coffee

Start, Rows: 0, 5

This query searches for documents containing the tag “coffee”. Along with only looking at the first 5 rows.

The screenshot shows the Solr Admin web interface in a browser. The left sidebar contains navigation links: Dashboard, Logging, Security, Core Admin, Java Properties, Thread Dump, megacollection (selected), Overview, Analysis, Documents, Paramsets, Files, Ping, Plugins / Stats, Query (selected), Replication, Schema, and Segments info.

The main panel is titled "Request-Handler (qt)" and shows the query configuration for the "/select" handler. The configuration includes:

- q: tags:coffee
- q.op: OR
- fq: (empty)
- sort: (empty)
- start, rows: 0, 5
- fl: (empty)
- df: (empty)
- paramset(s): Select paramset(s)...
- wt: (empty)
- ☒ indent on
- ☐ debugQuery
- defType: (empty)
- ☐ hl
- ☐ facet
- ☐ spatial
- ☐ spellcheck
- Raw Query Parameters: (empty)
- JSON Query: (empty)

The "Execute Query" button is visible at the bottom of the configuration panel.

The right panel displays the JSON response for the query. The URL in the address bar is `http://localhost:8983/solr/megacollection/select?indent=true&q.op=OR&q=tags%3Acoffee&rows=5&start=0&usef`. The JSON response is:

```
{
  "responseHeader": {
    "status": 0,
    "QTime": 1,
    "params": {
      "q": "tags:coffee",
      "indent": "true",
      "start": "0",
      "q.op": "OR",
      "rows": "5",
      "useParams": "",
      "_": "1770771865553"
    }
  },
  "response": {
    "numFound": 2,
    "start": 0,
    "numFoundExact": true,
    "docs": [
      {
        "id": "2",
        "collection_id": ["5048fde7c4aa917cbd4d8e13"],
        "websiteId": ["50295e80e4b096e761d7e4d3"],
        "enabled": [true],
        "starred": [true],
        "type": [1],
        "ordering": [3],
        "title": ["Food Blog"],
        "navigationTitle": ["Food Blog"],
        "urlId": ["food-blog"],
        "itemCount": [5],
        "updatedOn": [1454432709999],
        "pageSize": [20],
        "folder": [false],
        "dropdown": [false],
        "tags": ["pizza", "coffee"],
        "categories": ["category2"],
        "homepage": [false],
        "typeName": ["blog"],
        "synchronizing": [false],
        "fullUrl": ["/food-blog/"],
        "passwordProtected": [false],
        "typeLabel": ["blog"],
        "_version_": 1856787631105376256,
        "_root_": "2"
      },
      {
        "id": "1",
        "collection_id": ["5048fde7c4aa917cbd4d8e13"],
        "websiteId": ["50295e80e4b096e761d7e4d3"],
        "enabled": [true],
        "starred": [false],
        "type": [1],

```

Deliverable 3:

It was interesting working with a different dataset, because the JSON dataset that I had found had a lot more information. So, when I was doing the query, it was a lot longer, and there were more options that I could choose between on what to look up in the data. Solr does nicely and neatly lay out all the information in the structured data, so you can easily read/access it. You can easily filter, sort, and perform faceted searches within the Solr web or within the terminal. Experimenting with the queries in both examples, like searching by tags, filtering by Boolean fields, and sorting by numeric fields, shows how Solr can simply lay all this information quickly

and easily to read. After following the first example, it was pretty easy to replicate it with my own dataset.